

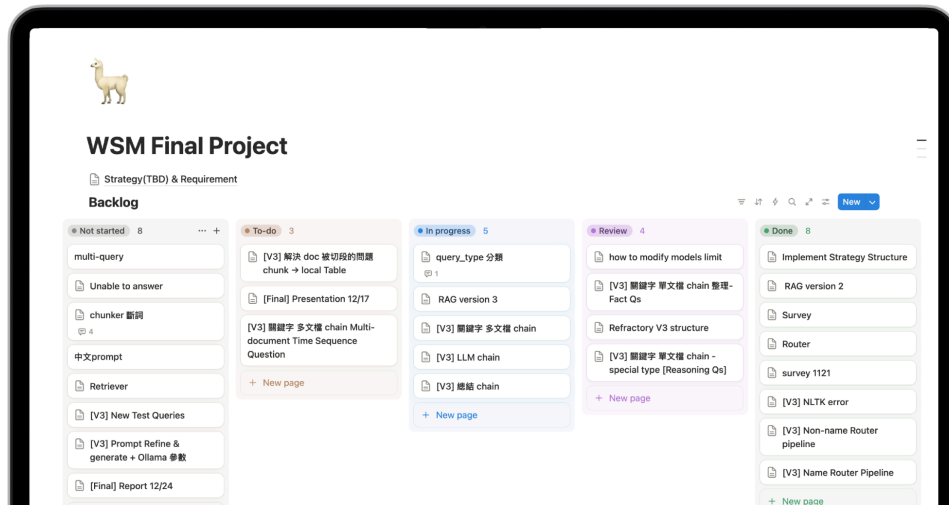
114-1 Web Search and Mining Final Project Report Team1

[GitHub](#)

Project Management & Collaboration

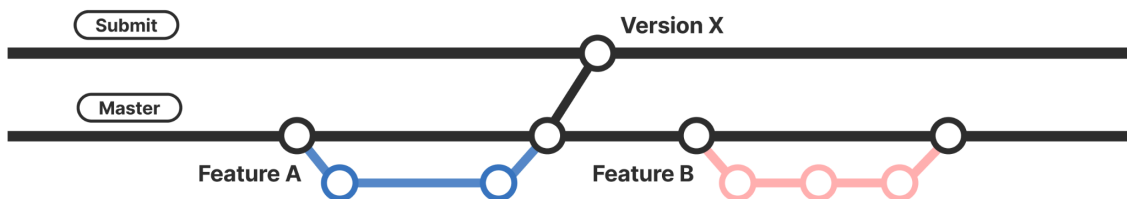
Agile Development

We maintained a **Product Backlog and Kanban** to track tasks and held **Online Meetings** twice a week to synchronize progress.



Code Management with Feature Branch Workflow

We utilized GitHub for version control and adopted a feature branch workflow. Each team member developed components on individual feature branches and committed changes independently. After implementation and verification, these changes were merged into the submit branch, which was then submitted to the leaderboard for evaluation.



Pre-development Research

Embedding Visualization Analysis

Through embedding visualization, we observed that queries can be clearly distinguished by domain, but query types are not well separated.

Analysis of Structures Across Query Types

To further investigate the difference across queries, we manually examined queries of different query types and analyzed their linguistic patterns. Our goal was to identify whether queries belonging to different types share similar structural characteristics, even when their semantic content differs.

Multi-Query & Query Decomposition

Multi-Query reformulates the same question from different perspectives to increase information coverage. For example, the question “缘源环保有限公司在 2017 年 1 月更新了哪些环保法律法规？” can be rewritten using terms such as *legal compliance announcements*, *policy adjustments*, or *regulatory changes*. By issuing multiple queries, the system reduces the risk of missing relevant information due to wording differences.

Decomposition, in contrast, breaks a complex question into smaller sub-questions, such as whether any updates occurred, whether they involved internal policies or external regulations, and which specific laws were affected. This step-by-step reasoning is effective for complex or multi-layered problems.

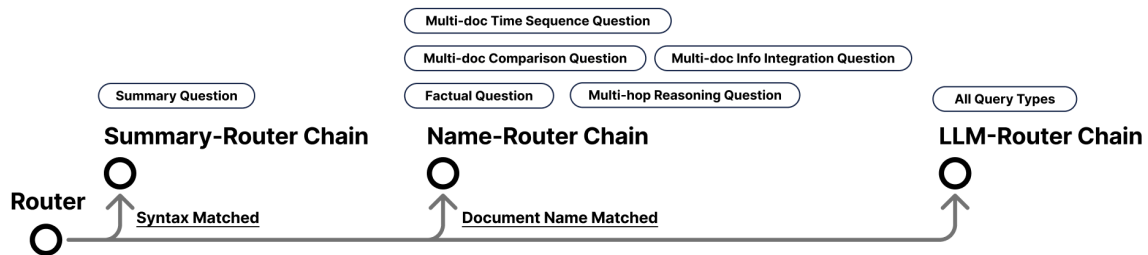
However, when the original question is simple, decomposition may introduce instability, as the model can invent unnecessary sub-questions. In such cases, multi-query or direct retrieval is often more stable and sufficient.

Two-step Chunking Strategy

First, documents were segmented into paragraph-level chunks based on newline boundaries(“\n”) to preserve broader contextual coherence. Subsequently, these larger chunks were dynamically segmented into sentence-level units. This approach facilitates the more precise retrieval of fine-grained factual information while simultaneously preserving the original narrative coherence.

System Structure & Core Implementation Details

System Structure Overview



The system processes queries individually and uses the **Router** to dispatch each query to the appropriate processing chain:

- **Summary-Router Chain**
 - Handles queries that match summary-related syntax.
 - This chain mainly addresses “Summary Questions”.
- **Name-Router Chain**
 - Handles queries that match exactly one or two documents.
 - This chain mainly addresses “Factual Questions” and “Multi-hop Reasoning Questions” with single document path; while multiple document matches path deal with “Multi-doc Info Integration Questions”, “Multi-doc Comparison Questions”, and “Multi-doc Time Sequence Questions”.
- **LLM-Router Chain**
 - Serves as a fallback for all other queries, which are handled directly by the LLM.

Build DB for Storing Documents

We utilized a lightweight database system, SQLite, to store our documents and preprocessed chunks. This design allows the system to access the database and retrieve the required document content efficiently and directly.

Use Faiss for Embedding

We utilized the **FAISS** library to build embedding indices in advance and serialize them into local files, enabling efficient and scalable similarity search during inference. This approach significantly reduces retrieval latency and allows repeated queries to be processed without rebuilding the index.

BM25 with Stemming and Stop Words

We employed the BM25 retrieval algorithm with stemming and stop word removal to improve term matching and retrieval effectiveness. Stemming helps normalize different word forms, while removing stop words reduces noise from high-frequency but low-information terms, resulting in more accurate and robust document ranking.

Keyword Matching (Document Name Matcher)

After analyzing the document corpus and example query datasets, we found that approximately 6,621 out of 6,710 queries contain the document name (hospital name, court name and company name). Therefore, we apply exact string matching to identify the relevant documents directly. When matching names are detected, the candidate set can be reduced from 108 documents per language to 1–2 documents, thereby improving retrieval accuracy.

Router : Multi-chain for Different Kinds of Queries

1. Summary Router Chain

We observe that queries containing keywords such as “Summarize” or “summarize” in English, as well as “总结” in Chinese, consistently correspond to summary-type questions.

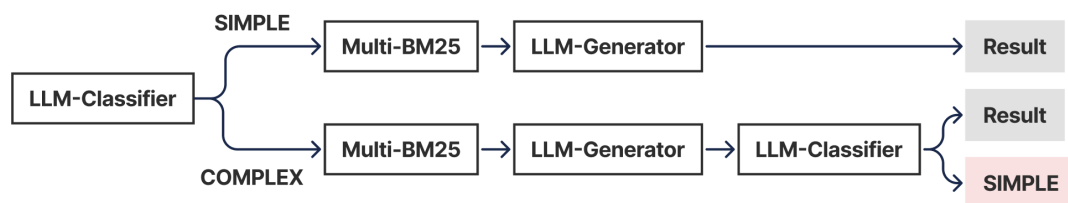
Unlike fact-seeking queries that focus on specific details, summary questions aim to obtain an overall understanding of the document.

In our approach, after the previous stage identifies the correct document, we directly provide the entire document as context to the LLM. The model is then instructed to identify relevant information within the document and generate the final answer.

2. Name Router Chain (one match, multiple matches)

After matching the document name in the query, we use this routing chain to dispatch the query to an appropriate processing path. Two main paths are defined based on the number of matched documents.

1. Single Document Match



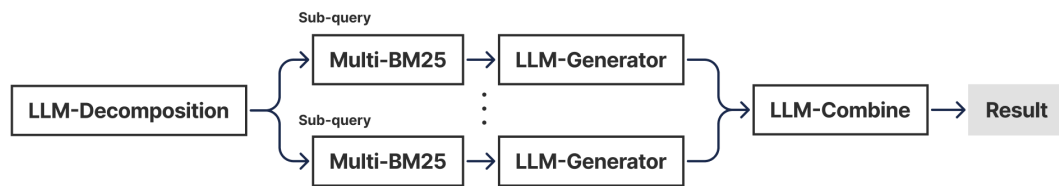
When exactly one document is matched, we first use an LLM-based classifier to categorize the query as SIMPLE or COMPLEX. Queries that ask about a single, specific entity (e.g., what, when, who) are classified as SIMPLE, whereas queries that require counting items, summing values, or listing multiple entities are classified as COMPLEX.

The SIMPLE path primarily handles “Factual Questions”, while the COMPLEX path is designed to address “Multi-hop Reasoning Questions”.

Since classification accuracy improves when using the query together with the generated answer, we apply an additional verification step for COMPLEX queries.

After generating an initial answer, we reclassify the query using the query–answer pair. If the result is reclassified as SIMPLE, the query is rerouted to the SIMPLE path to ensure that the final output better matches the query type.

2. Multiple Document Matches



When two or more documents are matched, we first use an LLM to decompose the original query into document-specific sub-queries.

For example, given the original query:

顶级购物中心和华夏娱乐有限公司分别在2018年和2017年完成了哪些重大资产收购？

the LLM generates the following sub-queries:

- "sub_query": "顶级购物中心在2018年完成了哪些重大资产收购？"
- "sub_query": "华夏娱乐有限公司在2017年完成了哪些重大资产收购？"

We then perform retrieval and generation for each sub-query independently, and finally combine or compare the results to produce a single answer to the original query.

When combining sub-queries, we attach the retrieved sentences to the prompt. Since these sentences may originate from different documents, we prepend each sentence with the corresponding document name to clearly indicate its subject.

For example, for the documents "**CleanCo Housekeeping Services**" and "**Retail Emporium**," the retrieved sentences are formatted as follows:

- **CleanCo Housekeeping Services:** In February, CleanCo underwent debt restructuring, reducing its liabilities by \$5 million.
- **Retail Emporium:** In August 2020, the company underwent debt restructuring, reducing liabilities by \$50 million.

Additionally, if the query contains explicit comparison cues (e.g., compare), we apply a compare-then-generate pipeline to produce a structured answer.

3. LLM Router Chain

For queries that cannot be handled by the **summary chain** or **name router chain**, the system falls back to the **LLM router chain**. In this stage, the router attempts to obtain additional information by expanding or reformulating the original query. We evaluated three query-expansion methods proposed in different papers.

Method 1: Add more information

This approach appends related nouns or concepts to the original query to improve recall.

Generated query example:

"When did significant ethical or integrity violations occur? significant ethics violations in business integrity breaches in organizations' ethics ..."

This method increases the likelihood of retrieving semantically related documents.

Method 2: Generate the answer first, then re-query with rationale

In this approach, the LLM first produces a tentative answer or reasoning, which is then used as a new query.

Generated query example:

“公司在2018年10月修订了什么政策？”

However, when the model is too small, it may fail to obtain correct facts, leading to biased reasoning and, consequently, biased outcomes.

Method 3: Direct retrieval with score-based feedback (Hybrid Retrieval)

The system performs direct retrieval, evaluates retrieval scores, and then requests a refined query to improve the score.

Generated query example:

“公司在2018年10月是否有修订任何特定的管理、合规或战略方面的政策？”

This method typically requires multiple retrieval rounds to provide feedback to the LLM iteratively.

Results of Expansion Query Test

	Overall Score	Retrieval Score	Generation Score
Method 1	35.52	<u>37.72</u>	33.33
Method 2	35.93	36.82	<u>35.05</u>
Method 3	35.46	<u>37.57</u>	33.35

Methods 1 and 3 achieved better retrieval performance, with scores of approximately **37.6**, outperforming Method 2 (**36.82**). In contrast, Method 2 performed best in answer generation, achieving a score of **35.02**, compared to around **33.3** for the other two methods.

For answer generation, we evaluated both the original query and the generated query as model inputs. In most cases, the generated query led to better results. However, when the generated query diverged significantly from the original intent—such as in Method 3, where the query is regenerated multiple times—using the original query produced better outcomes. This approach helps prevent the model from overfitting to the reformulated query and allows the generated answer to remain aligned with the original question.

Multiple BM25 Retrieval Strategy

While paragraph-level retrieval provides high recall, it suffers from low precision due to excessive irrelevant content. In contrast, our Multiple BM25 approach improves retrieval granularity by operating at the sentence level.

In the first stage, we retrieve larger sections using a relatively loose threshold to maximize recall. This step filters paragraphs that are broadly related to the query. The selected paragraphs are then further chunked into individual sentences. In the second stage, these sentences are retrieved using a stricter threshold to select only those with high lexical relevance to the query.

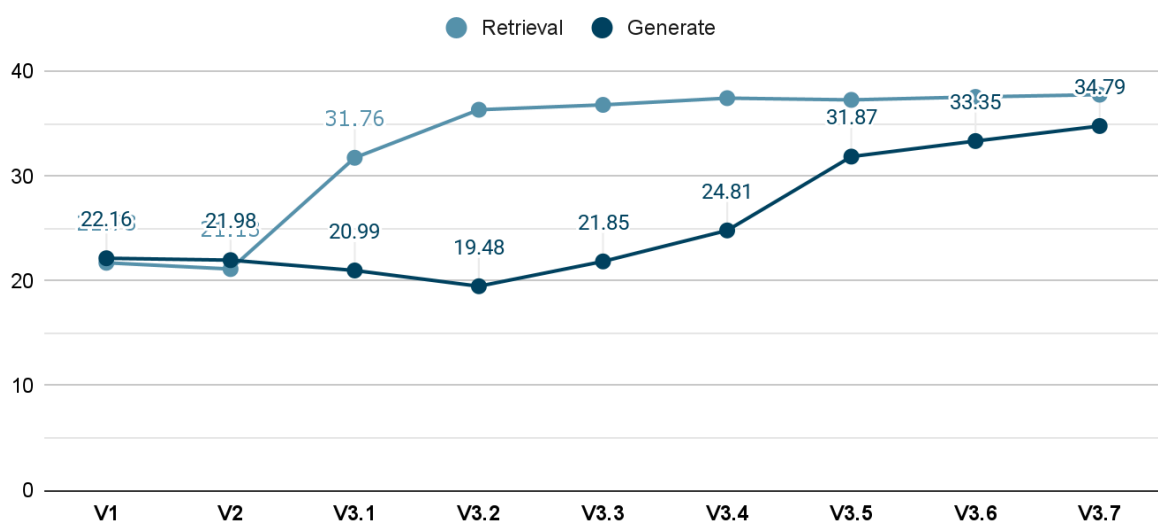
Since BM25 relies on term frequency, some sentences that merely contain the document name may receive high scores despite being semantically irrelevant. Conversely, relevant sentences may use pronouns instead of explicitly mentioning the document name. To improve retrieval accuracy, we remove the document name from sentences before performing sentence-level retrieval.

As an additional optimization, after generating an answer, we use the answer text to retrieve the selected sentences again. This retrieve-with-answer strategy is particularly effective for "Factual Questions."

Scores

From our mainly development progress, from V1 to final, our overall performance is like this:

	Overall Score	Retrieval Score	Generate Score
V1 Embedding Retrieval	21.94	21.73	22.16
V2 Single BM25 Retrieval	21.55	21.13	21.98
V3.1 Router + Multiple BM25 Retriever	26.37	31.76	20.99
V3.2 "Factual Question"	27.91	36.34	19.48
V3.3 "Multi-doc Info Integration Question"	29.33	36.81	21.85
V3.4 "Multi-doc Comparison Question"	31.13	37.45	24.81
V3.5 "Multi-hop Reasoning Question"	34.58	37.29	31.87
V3.6 LLM Router Chain	35.46	37.57	33.35
V3.7 Summary Router Chain	36.29	37.78	34.79



[V1, V2] Deal With High Recall & Low Precision

When using a single paragraph-level retrieval strategy, the system can capture the required information with high recall; however, it also retrieves a large amount of irrelevant content, resulting in low precision.

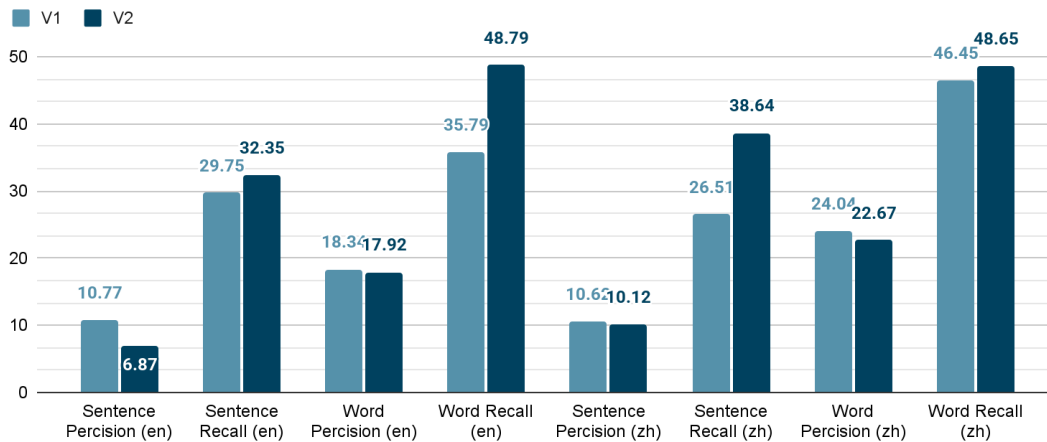


Figure: Comparing retrieval scores between V1 and V2, showing the precision and recall scores separated by en and zh.

[V3.1 - V3.4] Improving Retrieval Score Stage

After applying the Router and Multi-BM25 strategy, the retrieval score is significantly improved from 21.13 to 36.34 for V2 and V3.2. In particular, improved precision, along with a slight decline in recall on en, contributed to the higher overall retrieval score.

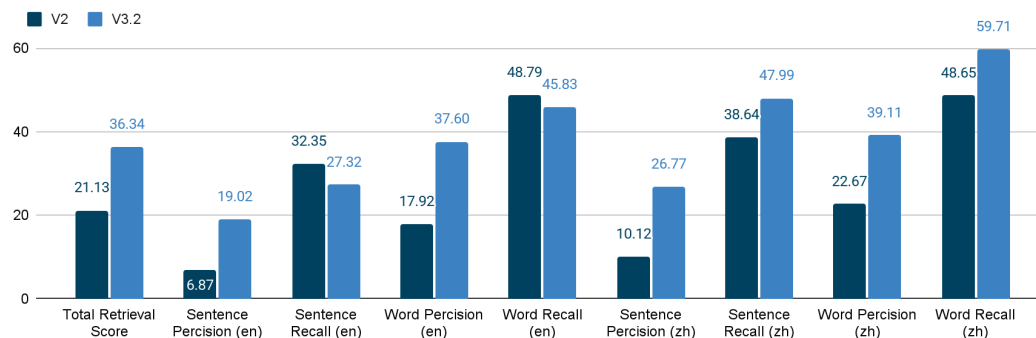


Figure: Comparing retrieval scores between V2 and V3.2, showing total retrieval score and precision and recall scores separated by en and zh.

[V3.5 - V3.7] Implement Question Specific Path & Fine-tune Our Prompts

From V3.5, we finally implement enough paths for the router to dispatch. For separating different kinds of queries, we designed different prompt templates tailored to each query type and corresponding processing chain in our system. This allows the LLM to better align its responses with the intended task. In addition, when no relevant information is retrieved, the LLM was instructed to explicitly respond with “Unable to Answer”, reducing unsupported or hallucinated outputs.

Findings and Observations

Prompt Design and LLM Configuration Matter Significantly

Even with accurate and explicit references, LLMs are not guaranteed to produce correct answers and may still generate incorrect responses. Therefore, we explicitly instruct the LLM using step-by-step guidance to steer it toward the desired output like:

Steps

1. Analyze the Question to understand what specific information is needed.
2. Scan the Reference Data to find the exact match.
3. If the answer is single and involves specific information (e.g., name, date, amount, location, project, event), answer with ONLY the specific requested information with the SAME format as the reference data. Do not use full sentences. Do not repeat the question or the context.

English Prompts Perform Better Even for Chinese Queries

Based on our experiments, despite Simplified Chinese queries and contexts, English prompts consistently yield better performance than Simplified Chinese prompts in retrieval-related settings.

Observed Limitations and Linguistic Variability in Comparison Generation

In some cases, LLMs tend to rewrite or reinterpret answers. In our compare-then-generate pipeline, when a comparison is required, we first prompt the LLM to generate a structured comparison in a concise form, such as “Retail Emporium + Net Profit 2020 + **Higher**.” This structured output is then used to generate the final natural-language answer.

Our expectation is that the structured representation guides the model to complete a consistent final answer using “**higher**” as the comparison wording. However, the LLM may be influenced by previously seen sequences, leading to subject selection bias. As a result, it may invert the perspective and produce an answer such as: “CleanCo Housekeeping Services' net profit of \$3.5 million was **lower** than Retail Emporium's net profit of \$50 million ...”. Although this statement is factually correct, it may receive a lower generation score because the evaluation expects the answer to follow the same subject framing and wording as the question.

Interestingly, the LLM may also generate responses like “Both CleanCo Housekeeping Services and Retail Emporium's net profits are high, ...”. While such outputs may appear imprecise, they are not strictly incorrect, as no explicit definition of “high” is provided. We find this phenomenon noteworthy, as it reflects how language encodes cultural and contextual interpretation. Even though LLMs mimic patterns of human reasoning, their outputs can exhibit variability and creative reinterpretation rather than strictly deterministic phrasing.

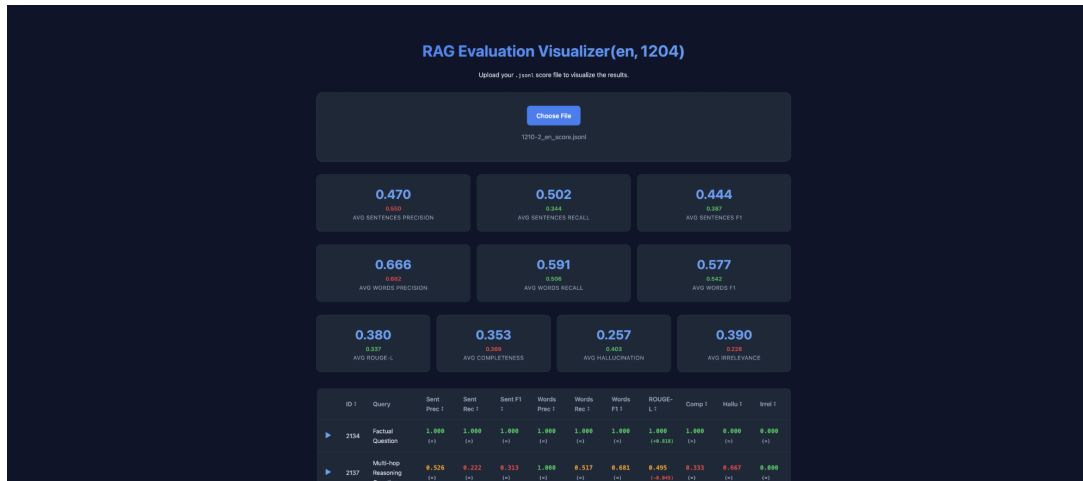
Reasoning Before Answer Generation Improves Accuracy

Since LLMs are fundamentally word-probability models, guiding the model to generate a rationale before producing a final answer can improve accuracy. When the model first generates a reason, it helps steer the probability distribution toward more relevant classifications. We adopt this approach in our LLM-based classifier within the Name-Router Chain, where reasoning-first generation leads to more accurate classification outcomes.

Development Tools Implementation

Score Dashboard

We developed a Score Dashboard to visualize experimental results in a tabular format, enabling straightforward comparison of results from each execution, including scores and their corresponding generated answers. This representation made it easier to evaluate and compare the performance of various strategies and configurations.



Prompt Playground

We set up a web-based playground for quick experimentation with the LLM, allowing us to test and optimize prompt design and configuration settings.

Future Works

Time Sequence Questions

Currently, time sequence questions are handled using the same processing path as comparison questions. However, this approach may not fully capture the time sequence relationships required for accurate reasoning. Future improvements may involve designing a dedicated handling strategy that explicitly models time order and time-based constraints.

Add Tags to Documents

We plan to enrich the document database with tags such as year, date, and time to better support time-related queries. While these tags provide useful metadata, further work is needed to design effective mechanisms for incorporating them into query processing and retrieval strategies.

Adoption of LLM Frameworks

We plan to explore the adoption of existing LLM orchestration frameworks, such as LangChain and LlamaIndex, further to modularize query routing, retrieval, and reasoning pipelines. These frameworks may help improve system scalability, maintainability, and enable more efficient experimentation with different chaining strategies.